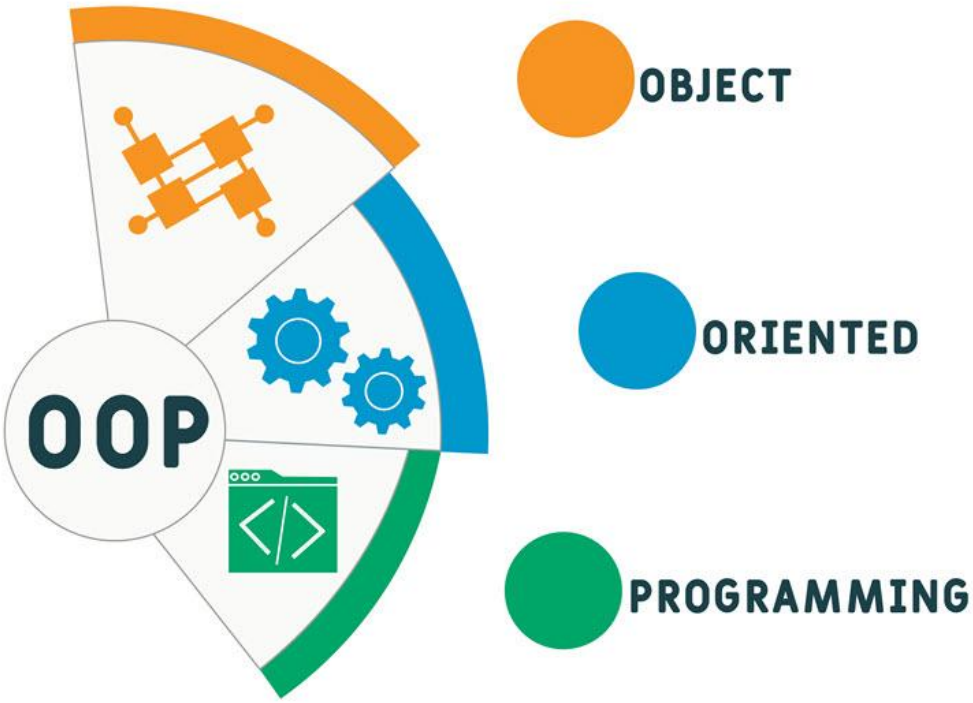
Instituto Superior de Engenharia de Coimbra
Instituto Politécnico de Coimbra

Relatório – Trabalho Prático de Programação Orientada a Objetos

2025/2026



Trabalho realizado por:
Rodrigo Glória – 2022150127
Martim Freitas – 2021145193

Índice

1. Introdução 3
1.1. Objetivos do Trabalho 3
1.2. Visão Geral da Solução 3
2. Arquitetura e Decisões de Implementação 4
2.1. Estruturas de Dados e Memória 4
2.2. Polimorfismo e Hierarquia de Classes 4
2.3. Padrão de Desenho *Command* 4
2.4. Persistência e "Deep Copy" 4
3. Descrição das Entidades Principais 5
3.1. Jardim e Solo 5
3.2. Jardineiro 5
3.3. Plantas Implementadas 5
3.4. Ferramentas 5
4. Persistência e Automatização 5
4.1. Sistema de Gravação de Estado (grava, recupera, apaga) 5
4.2. Execução de Scripts (executa) 6
5. Validação e Testes (Scripts) 6
5.1. Teste de Ciclo de Vida (morte_erva.txt) 6
5.2. Teste de Interação Ferramenta-Solo-Planta (excesso_agua_cacto.txt) 6
5.3. Teste de Vizinhança (sufoco_rosa.txt) 7
5.4. Teste de Metabolismo (roseira_fome.txt) 7
5.5. Teste de Combate e Área de Efeito (gadanha_vs_lirio.txt) 7
6. Conclusão e Análise Crítica 8

1. Introdução

Este relatório descreve o desenvolvimento de um simulador de ecossistema (Jardim) implementado na linguagem C++, no âmbito da unidade curricular de Programação Orientada a Objetos (POO). O projeto consiste na criação de um ambiente virtual onde interagem um agente principal (o Jardineiro), elementos biológicos (diversas espécies de Plantas) e ferramentas de manutenção, tudo gerido numa grelha bidimensional onde o tempo avança por instantes discretos.

1.1. Objetivos do Trabalho

O objetivo deste projeto foi a aplicação prática dos conceitos fundamentais de POO, nomeadamente o Encapsulamento, a Herança e o Polimorfismo, num cenário de simulação complexa.

Especificamente, o trabalho visou:

- Gestão Manual de Memória: Implementar estruturas de dados dinâmicas (matriz do jardim), exigindo uma gestão rigorosa de alocação e libertação de memória (new/delete) para evitar fugas de memória.
- Arquitetura Modular: Desenvolver um sistema robusto e extensível, utilizando padrões de desenho como o *Command* para o processamento de instruções do utilizador.
- Simulação de Regras Complexas: Modelar comportamentos biológicos distintos para cada espécie de planta (taxas de crescimento, reprodução, morte) e mecânicas de interação entre o jardineiro e o ambiente.

1.2. Visão Geral da Solução

A solução apresentada organiza-se em torno da classe Jardim, que atua como o motor central da simulação, gerindo uma matriz de objetos Solo. O sistema faz uso extensivo de polimorfismo para tratar de forma uniforme entidades distintas:

- As Plantas (Roseiras, Cactos, etc.) partilham uma interface comum, permitindo que o jardim processe o seu ciclo de vida sem conhecer os detalhes de cada espécie.
- As Ferramentas seguem o mesmo princípio, permitindo ao jardineiro utilizar qualquer utensílio de forma genérica.
- A interação com o utilizador é mediada por um Interpretador de comandos, que valida e executa as ações solicitadas, garantindo a robustez da aplicação face a *inputs* inválidos.

Ao longo deste documento, serão detalhadas as opções de arquitetura, as estruturas de dados utilizadas e as estratégias adotadas para cumprir os requisitos funcionais e não funcionais propostos no enunciado.

2. Arquitetura e Decisões de Implementação

A solução foi desenhada com base numa arquitetura modular, separando a lógica de dados, a lógica de simulação e a interface com o utilizador.

2.1. Estruturas de Dados e Memória

Para cumprir a restrição de não utilizar std::vector na representação do terreno, optou-se por uma matriz dinâmica de ponteiros:

- Implementação: A classe Jardim gere uma variável Solo** area. A alocação é feita em duas etapas no construtor (array de linhas e depois array de colunas) e a libertação é assegurada no destrutor da classe.
- Justificação: Esta abordagem permite um controlo total sobre o tempo de vida dos objetos Solo, garantindo que a memória é gerida de forma eficiente e explícita.

2.2. Polimorfismo e Hierarquia de Classes

O sistema faz uso extensivo de herança e métodos virtuais para tratar entidades heterogéneas de forma uniforme:

- Plantas: A classe abstrata Plantas define a interface comum (simular, clone, getTipo). As classes concretas (Roseira, Cacto, Erva, Lirio) implementam os comportamentos específicos.
- Ferramentas: Similarmente, a classe base Ferramentas permite que o Jardineiro e o Solo manipulem qualquer utensílio (Regador, Gadanha, etc.) sem conhecer o seu tipo específico.

2.3. Padrão de Desenho *Command*

Para o processamento de instruções, implementou-se o padrão Command.

- Existe uma classe base Comando com um método executar().
- A classe Interpretador mantém um vetor de instâncias de comandos (std::vector<Comando*>).
- Quando o utilizador insere texto, o interpretador percorre a lista e delega a execução ao objeto apropriado. Isto cumpre o princípio *Open/Closed*, facilitando a adição de novos comandos sem alterar o núcleo do interpretador.

2.4. Persistência e "Deep Copy"

A funcionalidade de gravar/recuperar o estado do jogo exigiu a implementação de um Construtor de Cópia robusto na classe Jardim.

- Em vez de copiar apenas os endereços de memória (*Shallow Copy*), o construtor percorre toda a grelha e cria novas instâncias das plantas e ferramentas utilizando o método polimórfico clone(). Isto garante que os "saves" são independentes do jogo atual.

3. Descrição das Entidades Principais

3.1. Jardim e Solo

O Jardim é o motor da simulação. O método simularInstante() é responsável por iterar sobre todas as células e invocar o comportamento das entidades presentes. A classe Solo atua como um contentor inteligente, gerindo a posse exclusiva de uma Planta* e uma Ferramenta*. O seu método setPlanta encarrega-se de apagar a planta anterior antes de atribuir uma nova, prevenindo fugas de memória na mecânica de invasão da Erva Daninha.

3.2. Jardineiro

O Jardineiro encapsula o estado do jogador: posição, inventário (std::vector<Ferramentas*> mochila) e limitações de fadiga (máximo de movimentos e ações por turno).

3.3. Plantas Implementadas

- Roseira: Lógica complexa de vizinhança e gestão de dois recursos (água e nutrientes). Morre por fome, sede ou superlotação.
- Cacto: Planta resistente que acumula recursos e morre por excesso de água.
- Erva Daninha: Planta invasora que se reproduz substituindo (matando) outras plantas vizinhas.
- Lírio (Exótica): Planta venenosa que causa dano ao jardineiro se houver interação incorreta (corte com tesoura).

3.4. Ferramentas

- Gadanha (Ferramenta Z): Implementa um algoritmo de corte em área que depende da orientação do jardineiro, afetando as células à sua frente.
- Tesoura, Regador e Adubo: Implementam as lógicas base de interação.

4. Persistência e Automatização

Para além da simulação interativa, o sistema implementa mecanismos robustos para guardar o estado do jogo e automatizar a execução de comandos, cumprindo os requisitos de administração do enunciado.

4.1. Sistema de Gravação de Estado (grava, recupera, apaga)

A funcionalidade de persistência permite ao utilizador criar "pontos de restauro" do jardim.

- Armazenamento em Memória: Optou-se por um sistema de armazenamento em memória RAM utilizando a classe estática GestorCopias. Esta classe mantém um contentor std::map<string, Jardim*>, associando um nome (ex: "save1") a um ponteiro para um objeto Jardim.
- Mecânica de Gravação (grava): Quando o comando grava é invocado, o sistema não se limita a guardar o ponteiro atual. Em vez disso, cria uma cópia profunda (*Deep Copy*)

integral do jardim atual (new Jardim(*atual)). Isto garante que o "save" é independente da simulação em curso; alterações futuras no jardim não afetarão a cópia guardada.

- Gestão de Recursos: Os comandos recupera e apaga asseguram a correta libertação de memória. Antes de substituir o jardim atual por uma cópia recuperada, o sistema faz delete do jardim antigo. Similarmente, ao apagar um save do gestor, a memória associada a esse jardim guardado é libertada.

4.2. Execução de Scripts (executa)

O comando executa <ficheiro.txt> permite carregar e executar sequências de comandos a partir de um ficheiro de texto externo.

- Reutilização do Interpretador: O comando 'executa' cria uma instância Interpretador temporário (interpretadorAux). O ficheiro é lido linha a linha e cada linha é injetada no método processarComando, simulando a entrada do utilizador.
- Utilidade Técnica: Esta funcionalidade foi essencial para a validação do projeto, permitindo a criação de cenários de teste complexos (como os scripts morte_erva.txt e gadanha_vs_lirio.txt) que seriam fastidiosos de digitar manualmente repetidas vezes.
- Tratamento de Erros: O sistema verifica a existência do ficheiro e termina a leitura graciosamente se encontrar o comando fim dentro do script, ou se chegar ao final do ficheiro.

5. Validação e Testes (Scripts)

Para garantir a robustez da simulação e o cumprimento das regras complexas do enunciado, foram criados *scripts* de teste específicos. Estes ficheiros de texto contêm sequências de comandos que isolam e validam comportamentos críticos.

5.1. Teste de Ciclo de Vida (morte_erva.txt)

Objetivo: Verificar se a Erva Daninha morre de velhice após 60 instantes.

- Cenário: Planta-se uma erva em aa.
- Execução: O comando avanca 59 faz o tempo passar. Ao avançar mais 1 instante, verifica-se através de lsolo aa que a planta desapareceu.
- Resultado: Confirma a implementação correta do contador de idade na classe Erva.

5.2. Teste de Interação Ferramenta-Solo-Planta (excesso_agua_cacto.txt)

Objetivo: Validar se o Cacto morre quando o solo tem excesso de água, e se o Regador altera corretamente o solo.

- Cenário: O jardineiro entra numa posição com o Regador equipado.
- Execução: Executam-se múltiplos avanca 1, o que ativa o Regador repetidamente, saturando o solo com água. De seguida, planta-se um Cacto.

- Resultado: O Cacto deteta a água excessiva no solo e morre após alguns turnos, validando a comunicação entre a classe Solo e a classe Cacto.

5.3. Teste de Vizinhança (sufoco_rosa.txt)

Objetivo: Testar a regra complexa de morte da Roseira por "sufocamento" (8 vizinhos ocupados).

- Cenário: Planta-se uma Roseira no centro (bb).
- Execução: Utilizam-se comandos para plantar Ervas Daninhas em todas as 8 posições circundantes (aa, ab, ac, ba, bc, ca, cb, cc).
- Resultado: Ao avançar o tempo, a Roseira morre, confirmando que o algoritmo de verificação de vizinhos na classe Roseira está a percorrer a matriz corretamente.

5.4. Teste de Metabolismo (roseira_fome.txt)

Objetivo: Verificar o decaimento dos atributos internos (Fome e Sede).

- Cenário: Uma Roseira é plantada num solo normal.
- Execução: O jardineiro apenas espera (avanca), e consulta-se periodicamente o estado com lplanta.
- Resultado: Observa-se a redução gradual dos nutrientes e água da planta até à sua morte, validando a lógica do método simular().

5.5. Teste de Combate e Área de Efeito (gadanha_vs_lirio.txt)

Objetivo: Validar a diferença entre a Tesoura (alvo único, curto alcance) e a Gadanha (área, longo alcance), bem como o perigo do Lírio.

- Fase 1 (Tesoura): O jardineiro entra na posição do Lírio com a Tesoura. O teste confirma que o jardineiro recebe dano (mecanismo de defesa da planta).
- Fase 2 (Gadanha): O jardineiro posiciona-se numa célula adjacente, vira-se para a planta (d - direita) e usa a Gadanha.
- Resultado: A Gadanha elimina o Lírio à distância (sem dano para o jardineiro) e corta também outra planta adjacente, validando o algoritmo de "Ataque em Área" baseado na orientação.

6. Conclusão e Análise Crítica

O trabalho prático foi concluído com sucesso, tendo sido implementados todos os requisitos funcionais. O simulador final permite uma interação fluida com o utilizador, suportando mecânicas complexas como ciclos de vida biológicos, gestão de inventário e persistência de estado.

A arquitetura adotada, baseada no encapsulamento forte e na delegação de responsabilidades, provou ser eficaz. O uso de classes abstratas para definir interfaces comuns permitiu tratar entidades heterogéneas de forma uniforme, simplificando a lógica do motor de simulação. A inclusão de funcionalidades de administração (execução de scripts e gestão de saves em memória) foi crucial para validar a robustez do código através de testes intensivos.

Em suma, este projeto permitiu não só aplicar a sintaxe de C++, mas também compreender a importância do planeamento arquitetural na construção de software modular e seguro.